Topics in Software Engineering – Software Reengineering

By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

Reverse Engineering

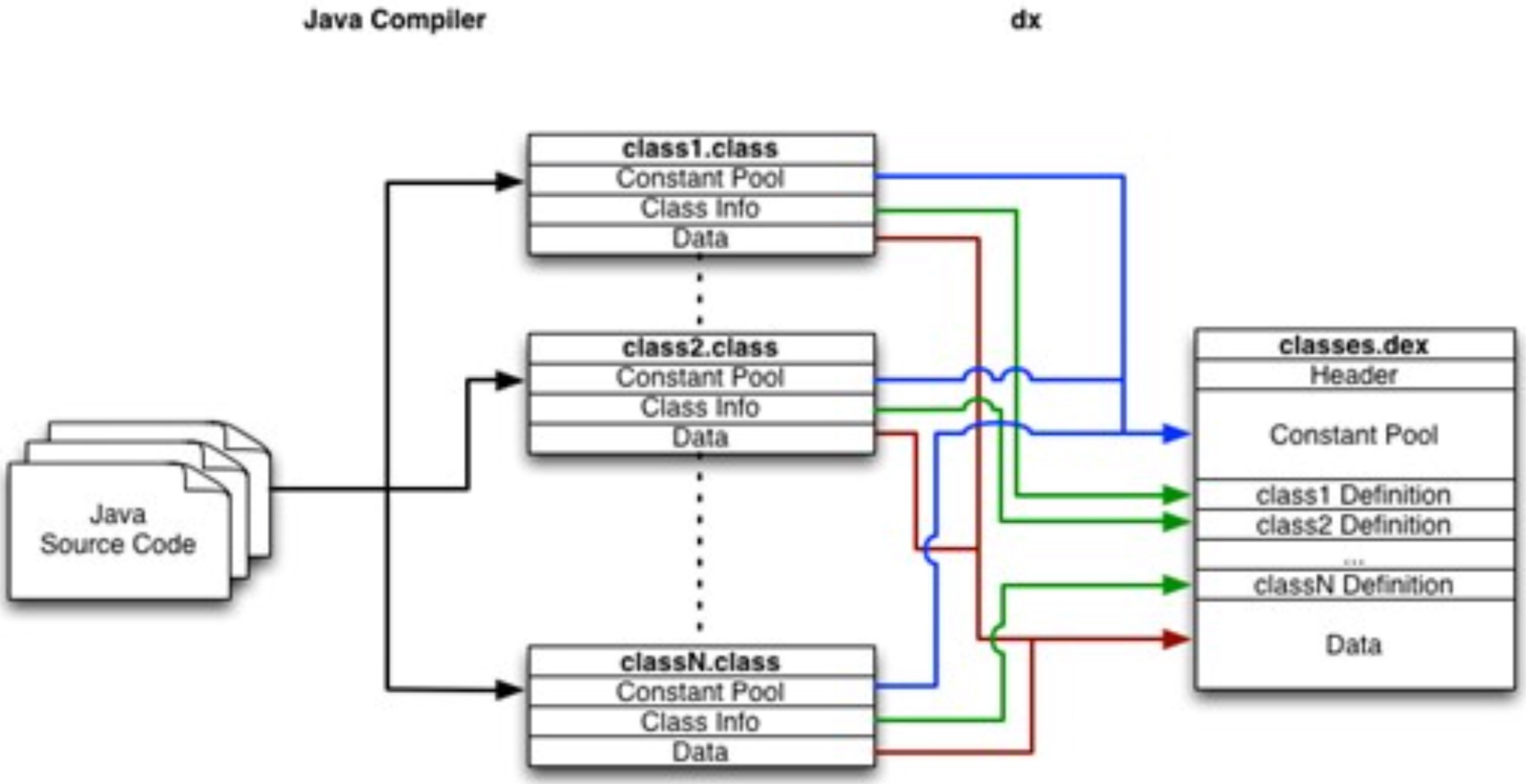
Concepts and Methods

Terminologies

- **Jar** (Java Archive) is a package file format typically used to aggregate many Java class files and associated metadata and resources.
- **Java bytecode** is the instruction set of the Java virtual machine.
- **APK** is the package file format used to distribute and install application software and middleware onto Google's Android operating system.
- **Obfuscation** is the obscuring of intended meaning in communication, making the message confusing, willfully ambiguous, or harder to understand.

Android Application Build Process

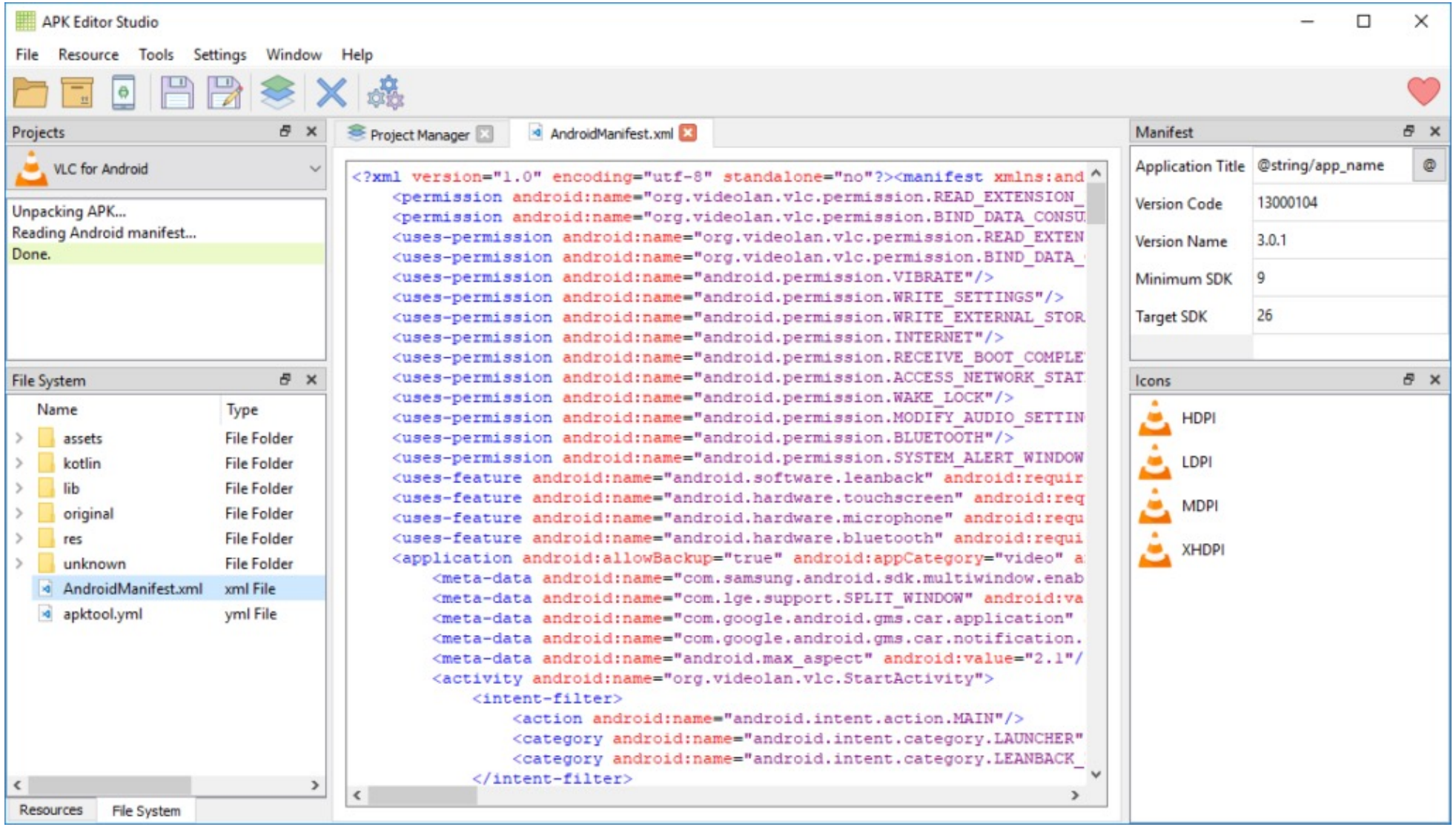
Cont..


Java Compiler
dx
class1.class
Constant Pool
Class Info
Data
class2.class
Constant Pool
Class Info
Data
classN.class
Constant Pool
Class Info
Data
Java
Source Code
classes.dex
Header
Constant Pool
class1 Definition
class2 Definition
classN Definition
Data

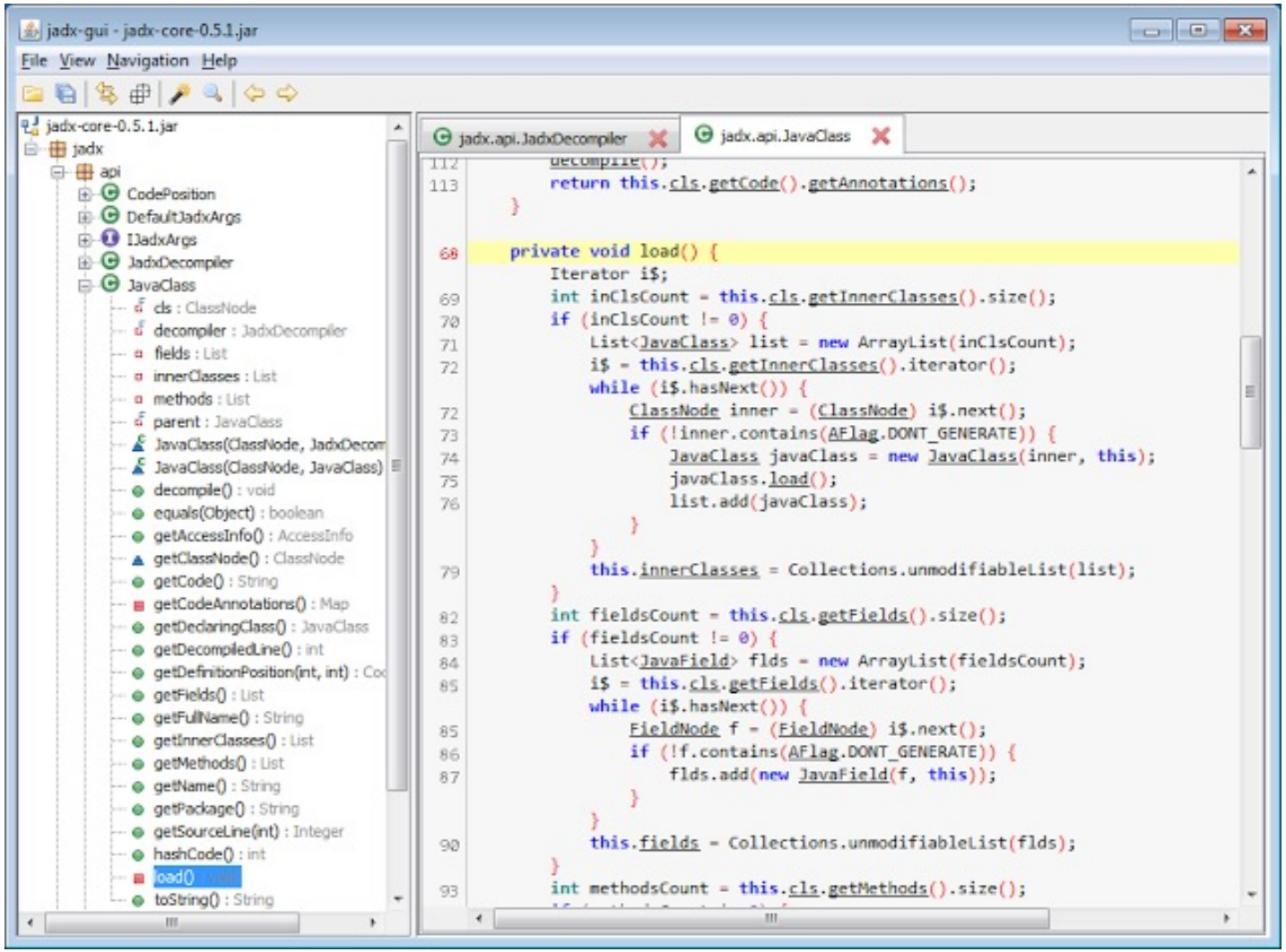
Tools

- Bytecode decompilation
  - JD-GUI
  - JAD Java Decompiler
  - Bytecode viewer
- Bytecode modification
  - Java Bytecode Editor
  - reJ
  - Javassist
  - Byte Buddy
- Bytecode debugging
  - Java ByteCode Debugger
  - Bytecode Visualizer

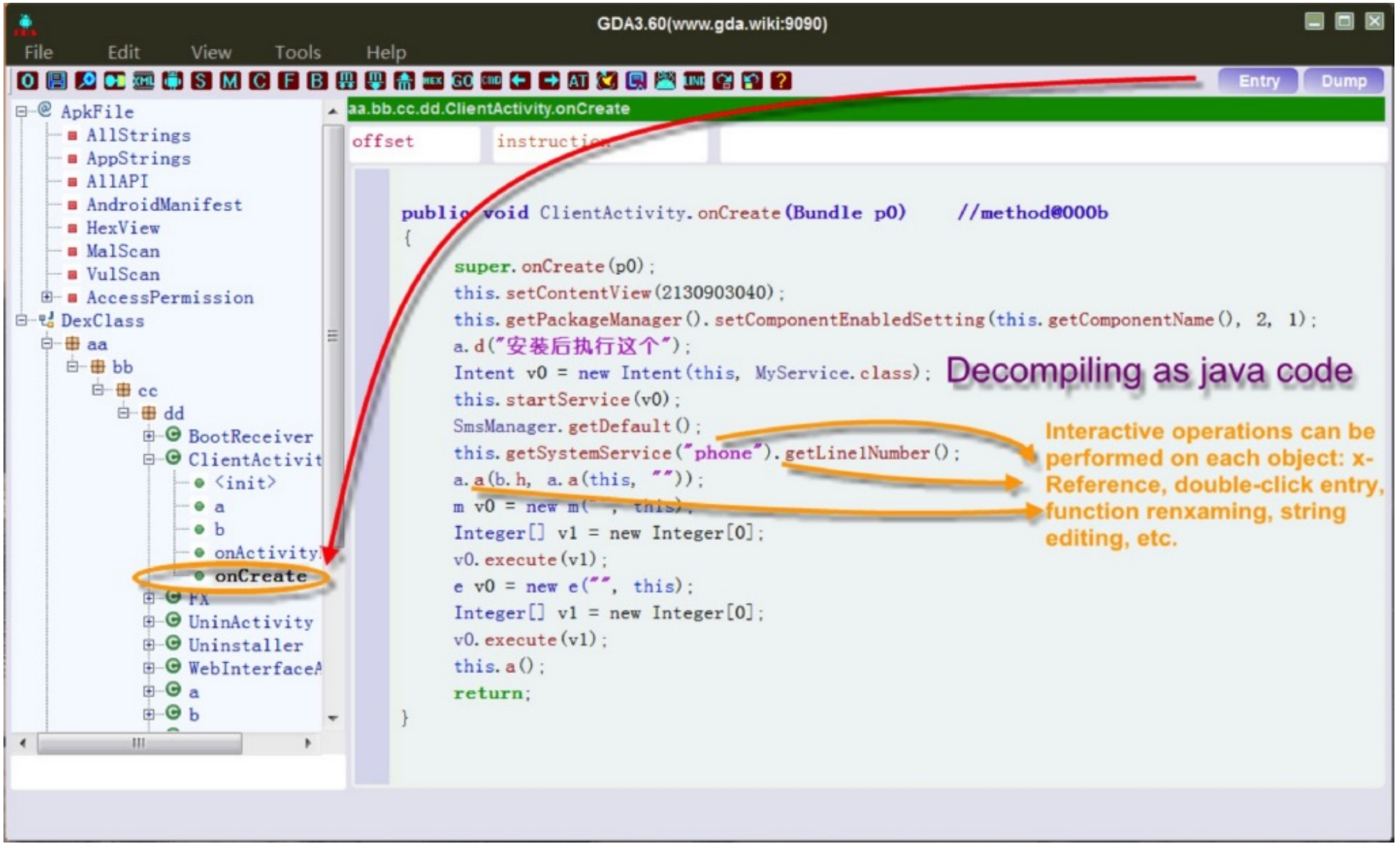
1: APK EDITOR STUDIO – Free, Open source & Cross-platform APK editor



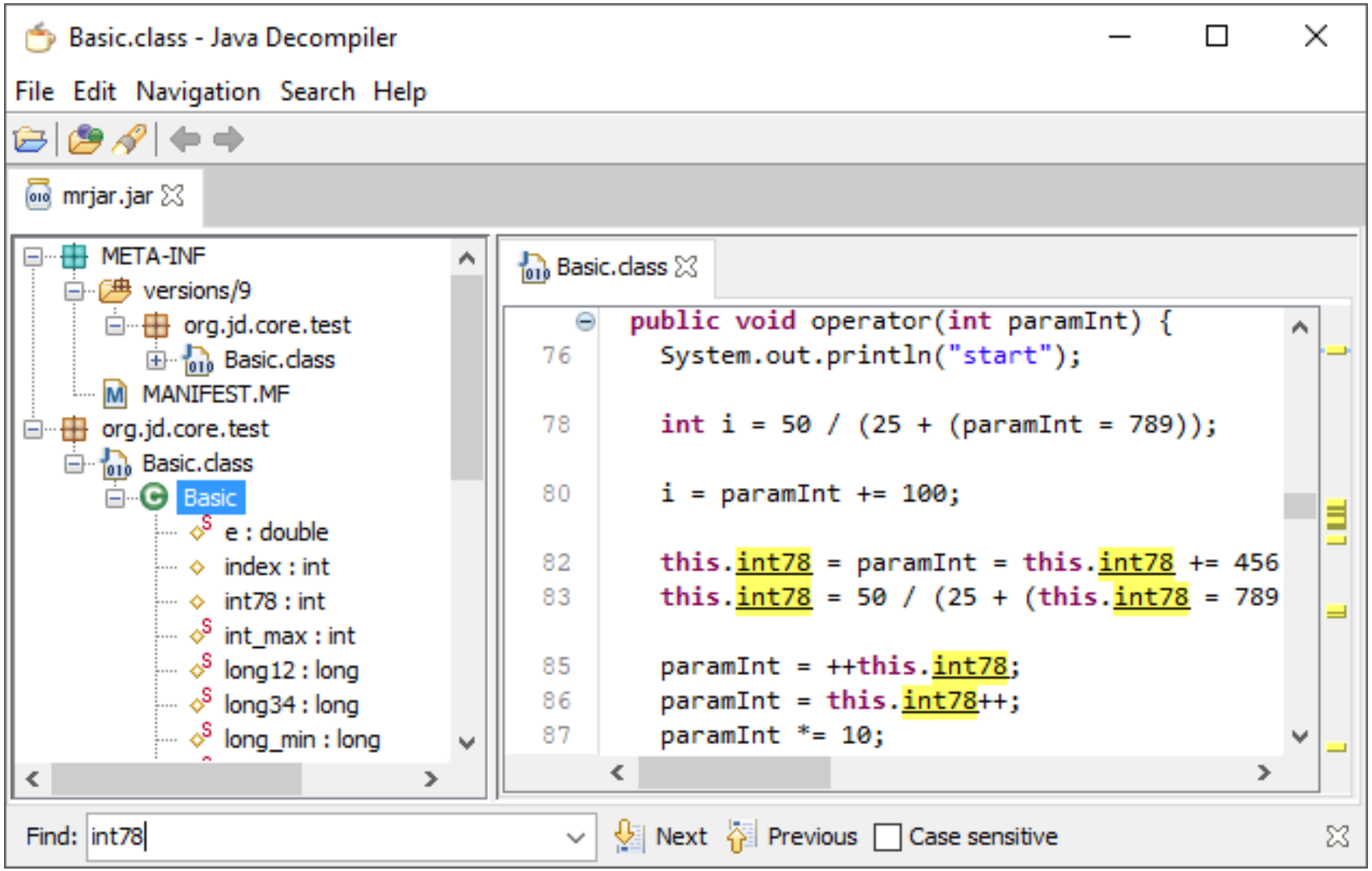
2: jadx – Dex to Java decompiler



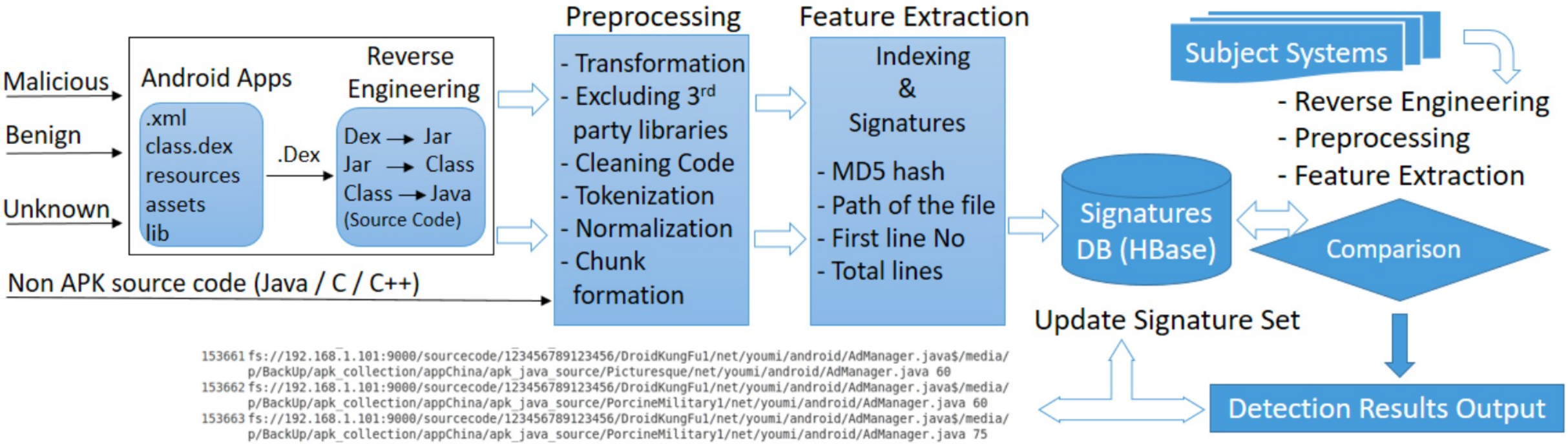
3: GDA – Android Reversing Tool



4: JD-GUI – Displays Java sources from CLASS files

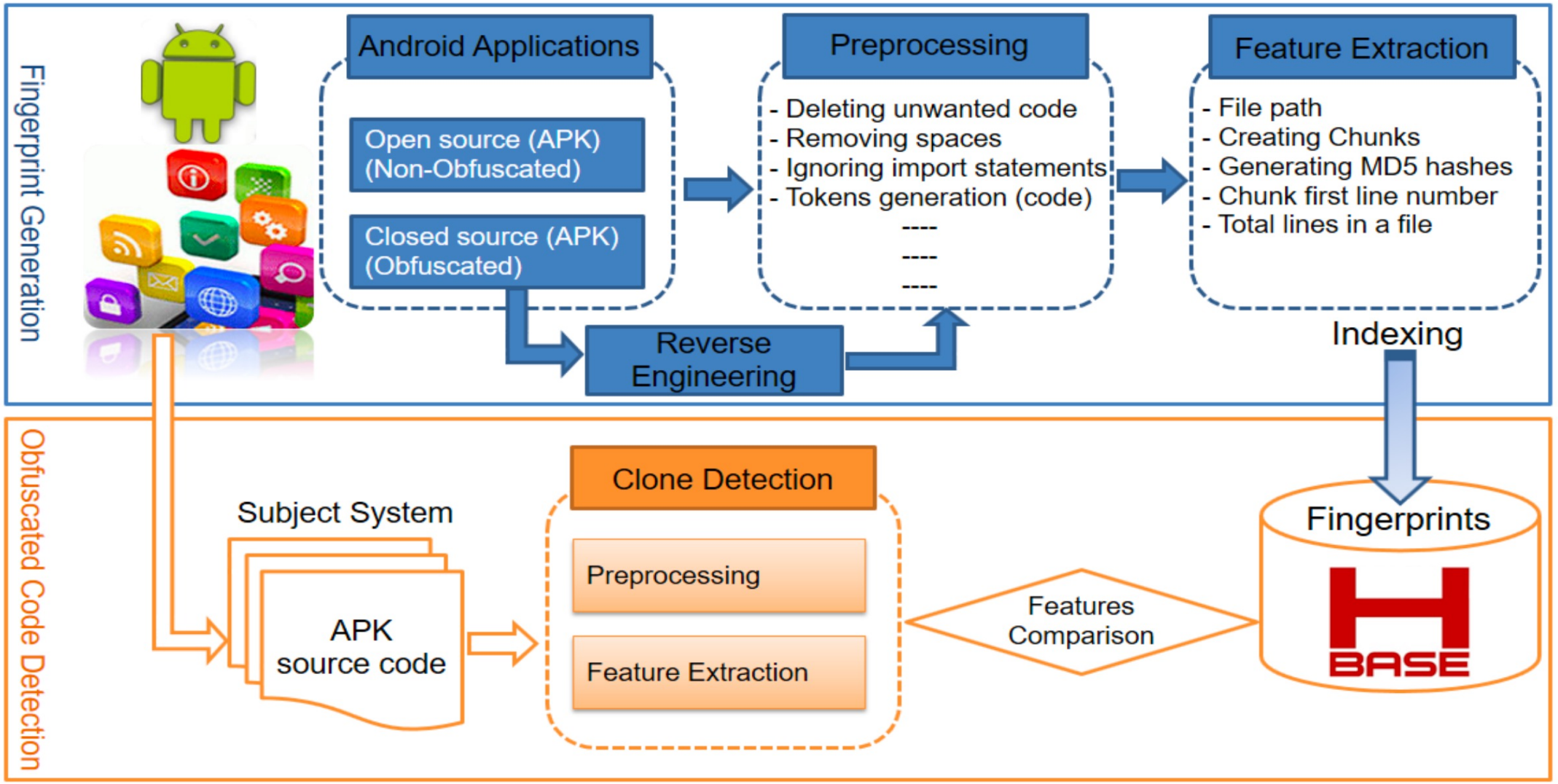


DroidMD : Android Malware Detection Main Architecture Using Reverse Engineering



DroidMD: an efficient and scalable Android malware detection approach at source code level
Published Online:July 9, 2021pp 299-321https://doi.org/10.1504/IJICS.2021.116310

Obfuscated Code in Reverse Engineering



Obfuscated code is identifiable by a token-base code clone detection technique

Published Online:November 21, 2022pp 254-273https://doi.org/10.1504/IJICS.2022.127132

Obfuscation

ALauncher.java

```java
import java.io.IOException;
public class ALauncher extends Service {
  public void onCreate() {
    //Toast.makeText(this, "made it to launcher",Toast.show();
    try {
        byte[] buff = new byte[250];
        FileInputStream fs = openFileInput("My_Last_Location");
        if(fs == null)fs = openFileInput("My_Last_Location2");
        fs.read(buff);
        fs.close();
        String st = new String(buff).trim();
        // Toast.makeText(this, st, Toast.LENGTH_LONG).show();
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(st));
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        startActivity(intent);
        } catch (FileNotFoundException e) {
           Toast.makeText(this, "No data", Toast.LENGTH_LONG).show();
           e.printStackTrace();
        }
    }
}
```

ALauncher_obfuscated.java

```java
import java.io.IOException;
public class abcdefghijklmnopqrstuvwxyz extends Service {
  public void xyzzzzzzzzzzzzzz() {
    //Toast.makeText(000000000, "6d616465206974207461206c61756e63686572";
    try {
        byte[] xxxxxxxxx = new byte[250];
        FileInputStream ffff = openFileInput("4d795f4c6173745f4c6f636174696f6e");
        if(ffff == null)ffff = openFileInput("4d795f4c6173745f4c6f636174696f6e32");
        ffff.read(xxxxxxxxx);
        ffff.close();
        String zzzzzzzzzzzzzzzzzzzzzzzzzzzzzz = new String(xxxxxxxxx).trim();
        // Toast.makeText(this, st, 546f6173742e4c454e4754485f4c4f4e47).show();
        Intent yyyyyyyyyyyyyyy = new Intent(Intent.ACTION_VIEW, Uri.parse(st));
        yyyyyyyyyyyyyyy.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        startActivity(yyyyyyyyyyyyyyy);
        } catch (FileNotFoundException e) {
           Toast.makeText(this, "4e6f2064617461", Toast.LENGTH_LONG).show();
           e.printStackTrace();
        }
    }
}
```

Chunk generation, hashing, and indexing of fingerprint values

```
int[] data;
int size;
public boolean binarySearch(int key)
{
int low = 0;
int high = size - 1;
while(high >= low)
{
int middle = (low + high) / 2;
if(data[middle] == key)
return true;
if(data[middle] < key)
{
low = middle + 1;
mActivity = null;
}
if(data[middle] > key)
{
high = middle - 1;
}
}
return false;
}
```

```
id0 id1;
id0 id1;
id0 id1 id2 id3
{
id0 id1=0;
id0 id1=id2-0;
while id0 >= id1
{
id0 id1=id2+id3/0;
if id0 id1==id2
return true;
if id0 id1<id2
{
id0=id1+0;
Id0=null
}
if id0 id1>id2
{
id0 =id1-0;
}
}
return true;
}
```

Chunk1:
9285EBD2445B5A59A313FA033AC5F2A5
Chunk2:
68D8FFD71D3DF3628663F513C34E2097
Chunk3:
E333D4F0FD439DA4AB0720146381D4DD

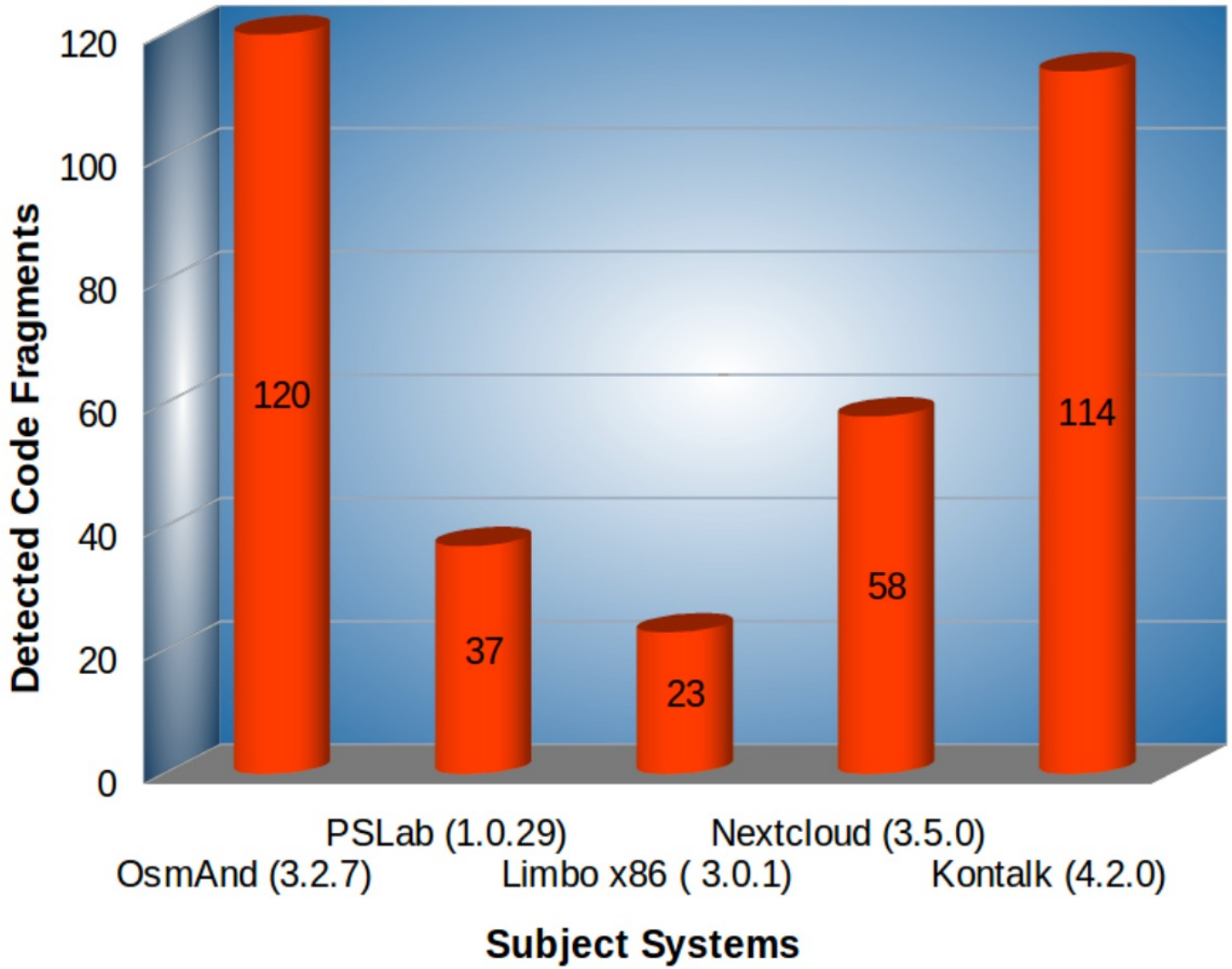
Chunk12:
862B0BCD8C9D66C7F3519EF4247C6297
Chunk13:
27354D55E43C1618DF83D370B489B5A2

HBase

D4B2758DA0205C1E0AA9512CD188002A
08C6A51DDE006E64AED953B94FD68F0C
BC7B36FE4D2924E49800D9B3DC4A325C
9C722330351000F2B4CECD6CAF813BD8
540FB4A8A362F8138D445F1498F50FA2
2E6B0DD0887BEB469976914DF5B87621
92B5BC04D51B144C177D058C6E9A98FC
6226F7CBE59E99A90B5CEF6F94F966FD
6FAC3AB603BB3FB46E4277786393194C

Obfuscated Code Generation Through Proguard

Obfuscated Fragments Retrieved Against Five Subject Systems



Android Reverse Engineering Tools

- APKtool: A powerful tool for reverse engineering APK files. It can decode application resources to their nearly original form and rebuild them after making code modifications.
- JADX: This is a command-line and graphical tool that can decompile DEX (Dalvik Executable) files and convert them to readable Java source code
- Dex2jar and JD-GUI: dex2jar is a tool used to convert DEX files to java JAR files and then using JD-GUI, which is a Java decompiler that can be used to view the Java source code
- Radare2 (also known as "r2"): This is a free and open-source reverse engineering framework that can analyze, modify, and decompile Android applications.
- Strings: A simple utility that extracts and displays printable strings from a binary file. It can pull strings from Android APK files and be a valuable tool for reverse-engineering Android applications.

A curated list of awesome Android Reverse Engineering training, resources, and tools.

Static Analysis Tools

- QARK - An open-source tool developed by LinkedIn for automatic Android app vulnerability scanning, including identifying potential security issues such as SQL injection, insecure data storage, and more.
- Quark Engine - The goal of Quark Script aims to provide an innovative way for mobile security researchers to analyze or pentest the targets. Based on Quark, we integrate decent tools as Quark Script APIs and make them exchange valuable intelligence to each other.
- MobSF - An open-source mobile app security testing framework that supports static and dynamic analysis of Android apps for vulnerabilities and privacy issues.
- AndroBugs Framework - An open-source framework for analysing and scanning Android apps for security issues, including static and dynamic analysis capabilities.
- ☆ imjtool - Firmware unpacking tool applicable to the widest variety of vendors and formats.
- Android Studio - Useful if you don't have a JEB licence and want to open a decompiled (via JADx) app into a proper IDE.
- ☆ APK Dependency Graph - An APK class dependency visualizer. Useful for attack surface mapping.
- disarm - A simple command line utility that takes as an argument a 32-bit hexadecimal number, and parses it as an ARM-64 instruction, providing the disassembly.
- COVA - COVA is a static analysis tool to compute path constraints based on user-defined APIs.
- DIS{integrity} - A tool for analysing Android APKs and extracting root, integrity, and tamper detection checks.

- https://github.com/user1342/Awesome-Android-Reverse-Engineering

Thanks for your attention!

Any Question?

Email me on : junaidakram@cuilahore.edu.pk